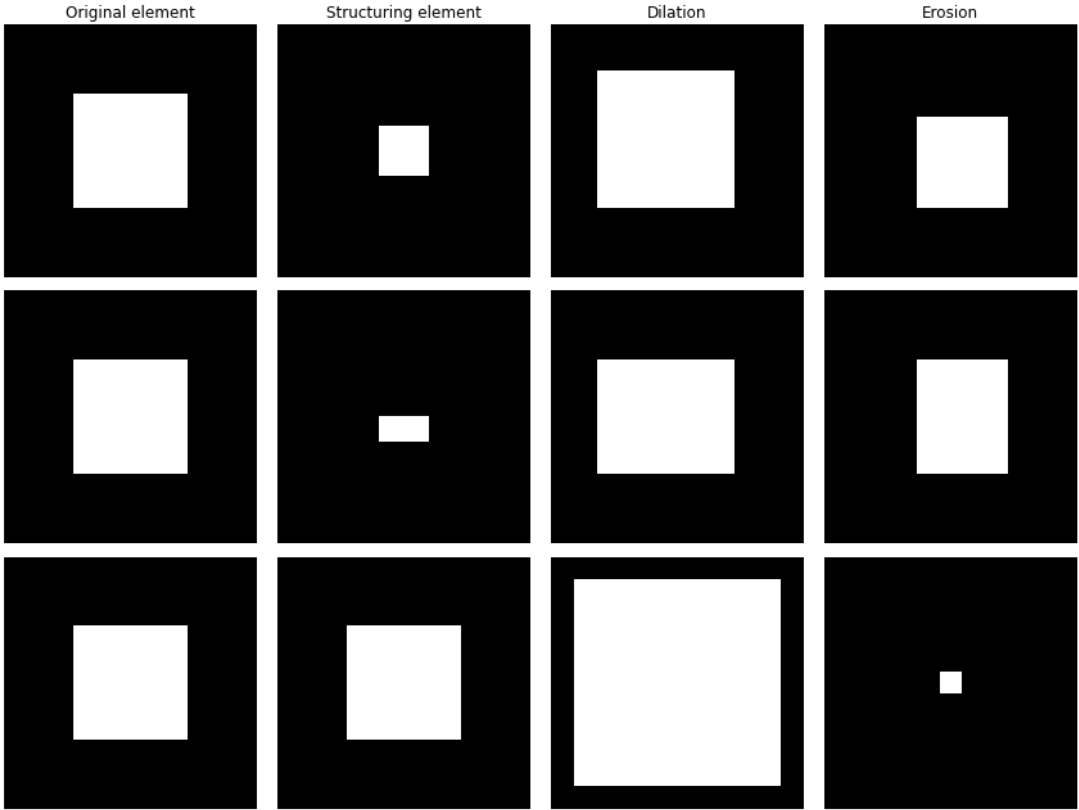
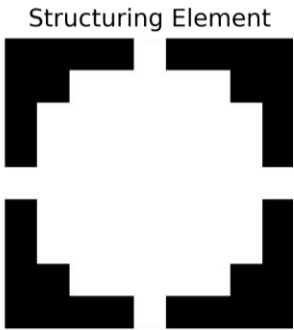
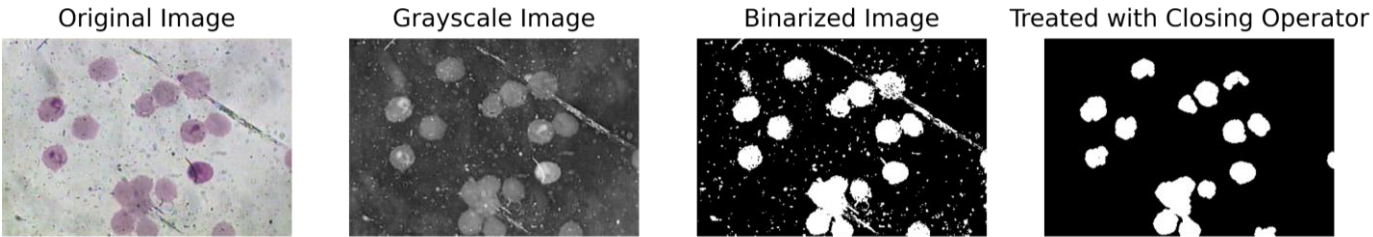




Morphological Operations



Presented by **Soriano, Edneil E. Jr.**

S.N. 202002420

Objectives

- To understand how morphological operations in Set Theory behave – particularly dilation and erosion.
- To determine how morphological operations can be used to clean up binary images in preparation for feature extraction.

Dilation and Erosion

Consider two sets A and B (these will be the images later), where z is a position in integer space. Then, the erosion and dilation of these two sets is defined below:

$A \oplus B = \{ z \mid (\hat{B})_z \cap A \neq \emptyset \}$ Dilation

$A \ominus B = \{ z \mid (B)_z \subseteq A \}$ Erosion

These morphological operations can be used to clean up binarized images due to their ability to modify images. In our case, we treat the set A as our image while the set B is called a *structuring element*. The structuring element is commonly a simple shape and it dictates the behavior of both operations.

The effects of both operators on an image is listed on Table 1.

Table 1. Effects of dilation and erosion on an image A and a strel B

Dilation	Erosion
expand/elongate A in the shape of B	reduce A by the shape of B
thicken bridges between blobs	remove bridges between blobs
decreases size of holes in blobs	increases size of holes in blobs
increases white region in image	erodes boundaries of foreground object

In order to implement these operations the following algorithm can be used: A “hit” occurs when SOME pixels of B overlaps A while a “fit” occurs when ALL pixels of B overlaps A. We then move B everywhere on the image (no rotation). We put a 0 or a 1 at B’s origin depending on the ff rules

Dilation

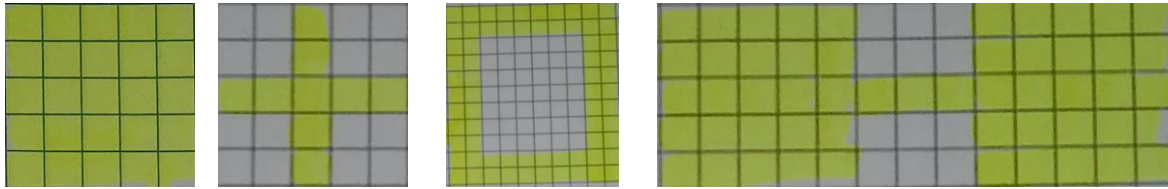
if hit occurs:
 put 1 in origin of B
else:
 put 0

Erosion

if fit occurs:
 put 1 in origin of B
else:
 put 0

Dilation and Erosion

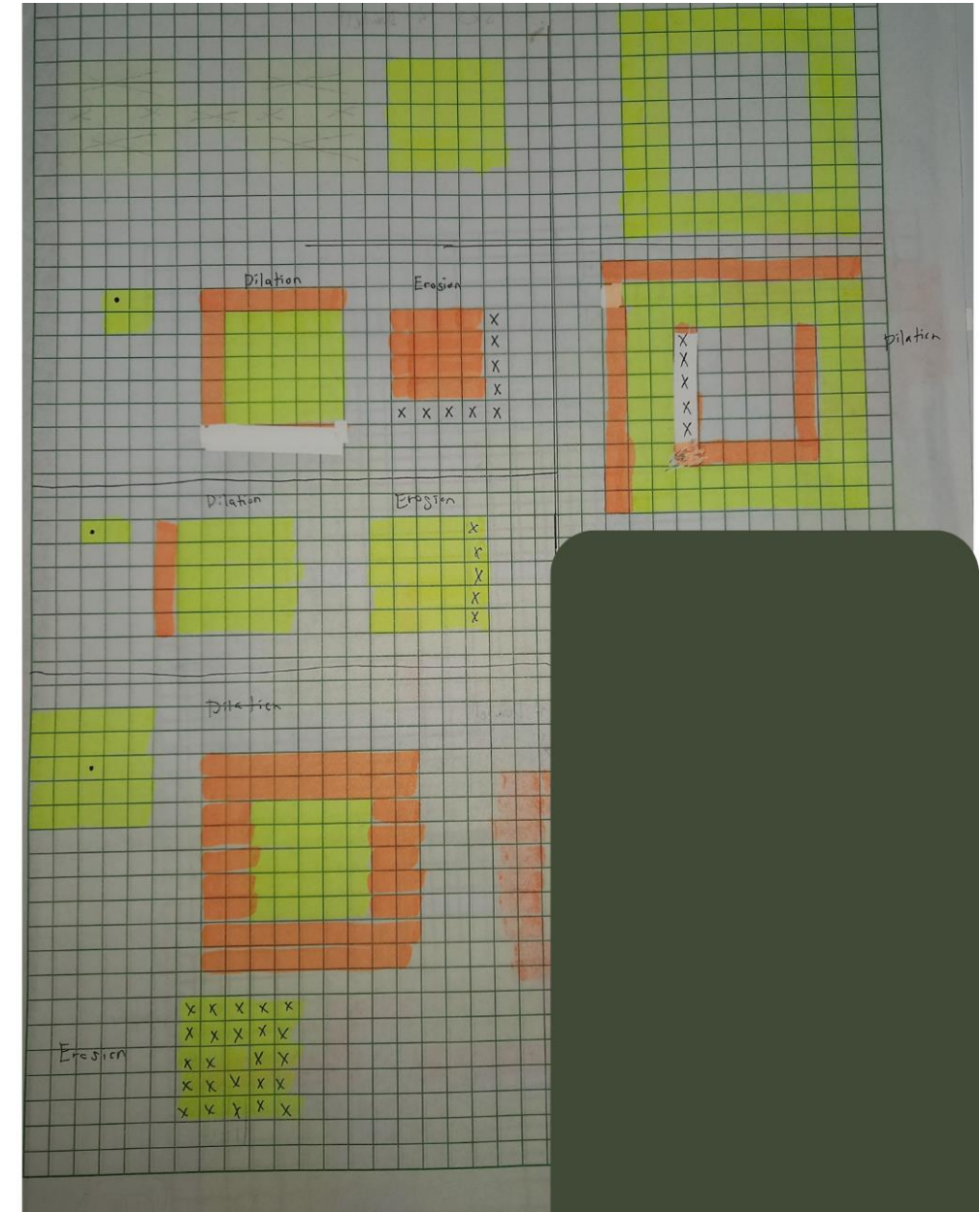
Here, we apply the described algorithm by hand by taking the following images:



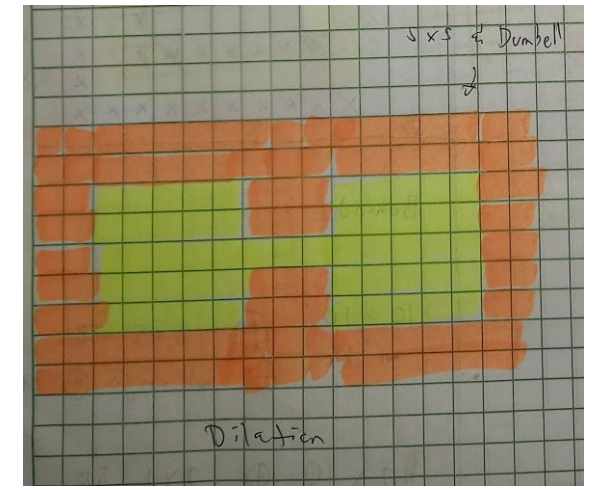
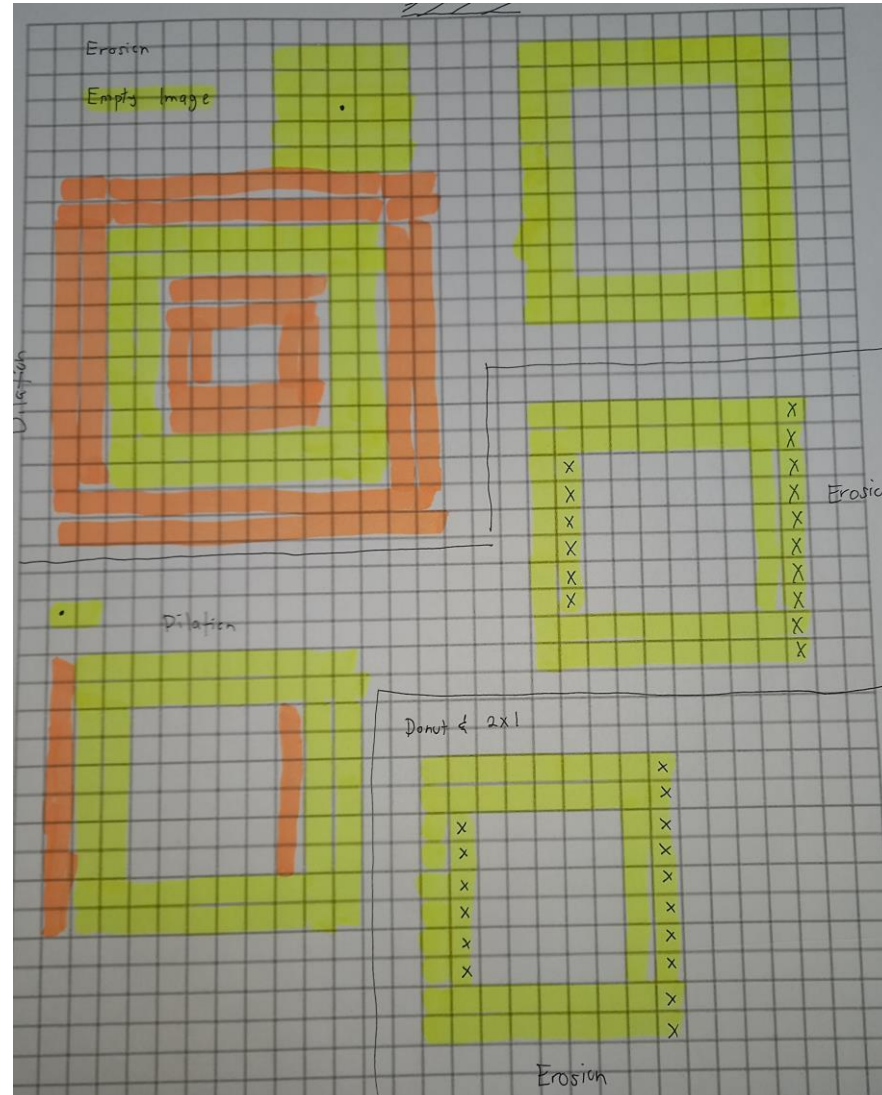
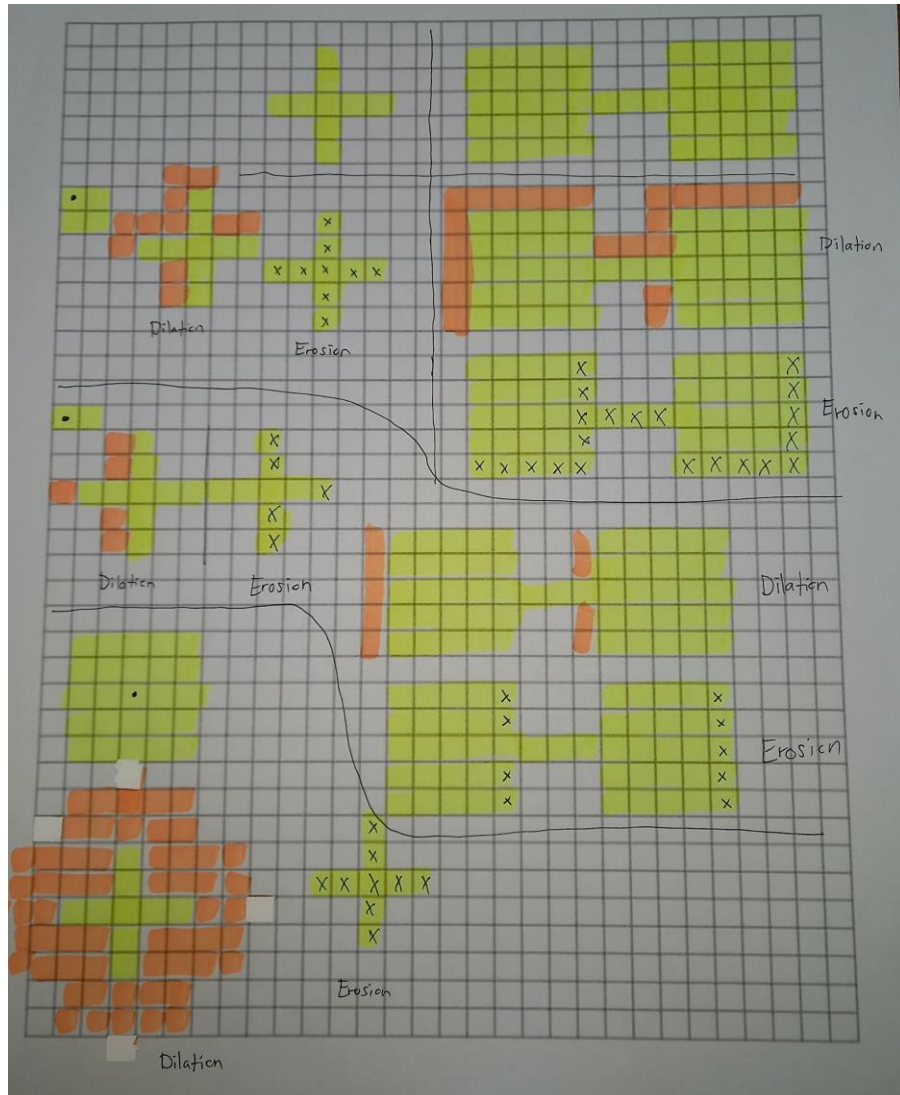
And applying both dilation and erosion, for each of the following structuring elements:



Note the dots used to represent the origin of each structuring element. My raw calculations are shown on the left and on the next slide. An “x” denotes deletion of the pixel in question.



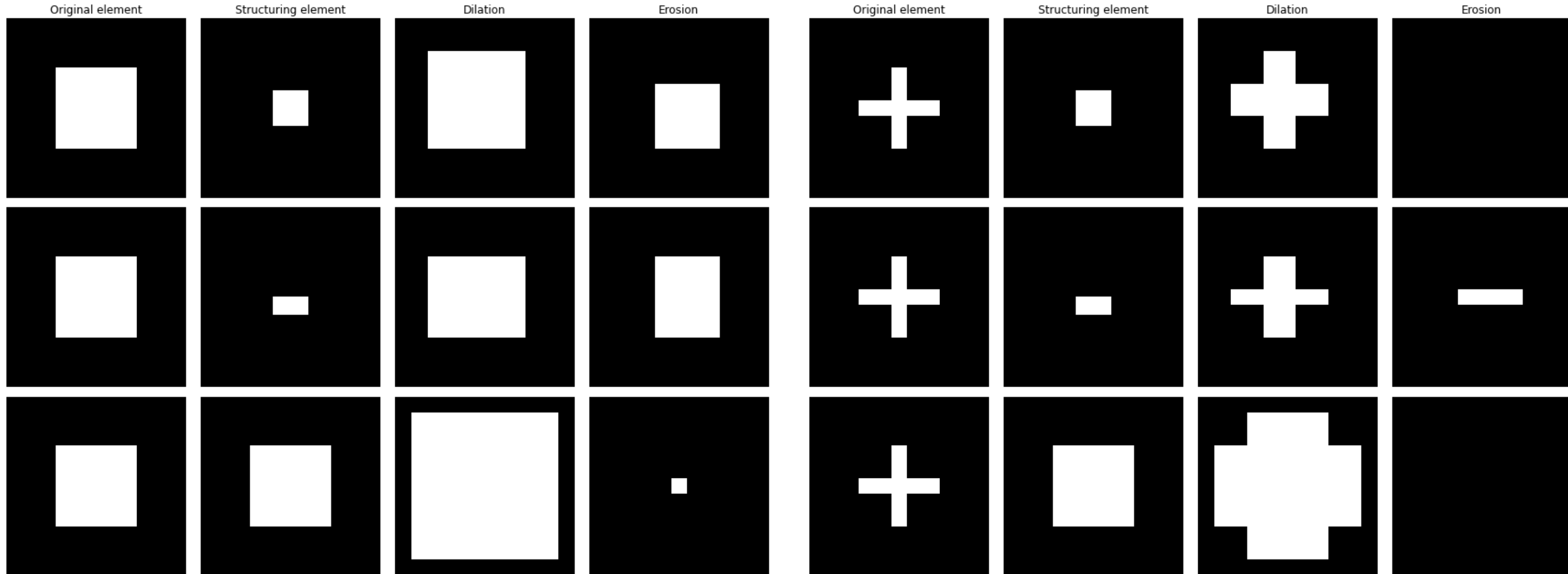
Dilation and Erosion



We see that the predictions made in Table 1. were accurate and reflected the changes made to the images in my manual calculations. This is why we chose the donut and dumbbell as test images – to test the effects of the two operators on holes and bridges.

To confirm, we now do the same procedure in Python using the scipy library as shown in the next slides.

Dilation and Erosion



We can see that the dilation operator expands an image while the erosion operator shrinks it. This behavior is still evident when corners are prevalent in an image such as in the cross. Lastly, the structuring element used determines the orientation of the shrinking/expanding .

Dilation and Erosion

Original element

Structuring element

Dilation

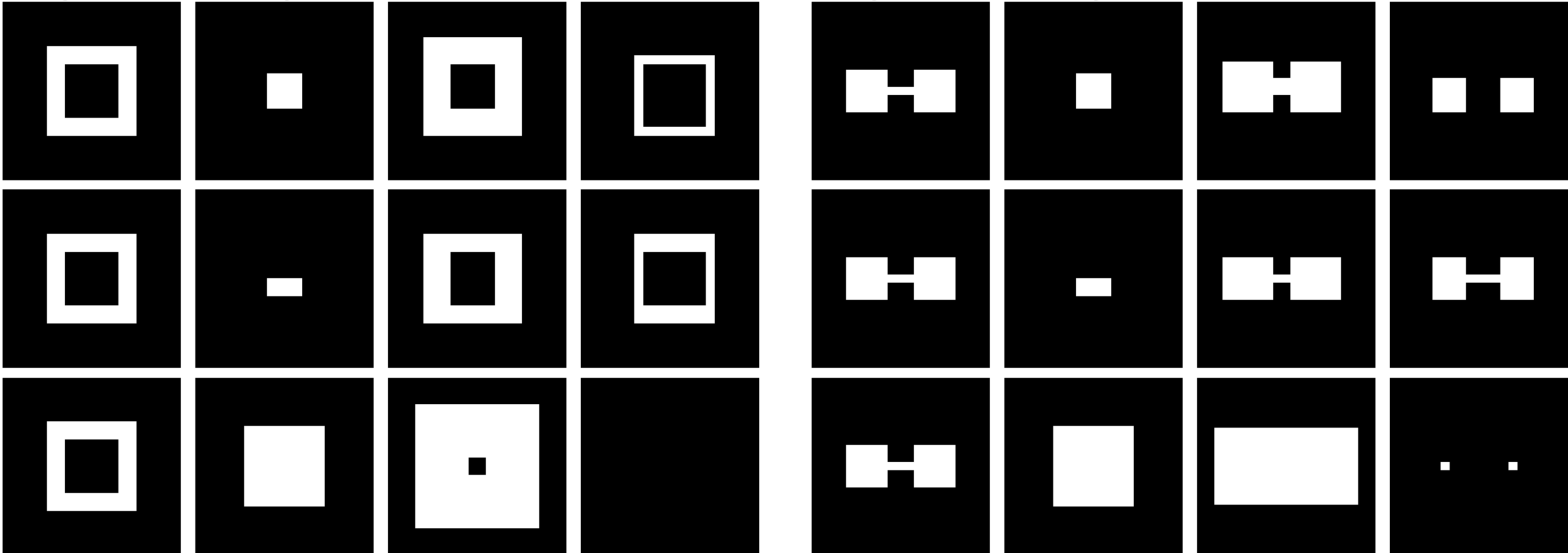
Erosion

Original element

Structuring element

Dilation

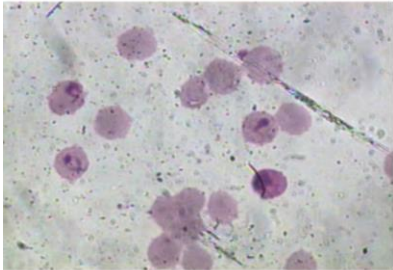
Erosion



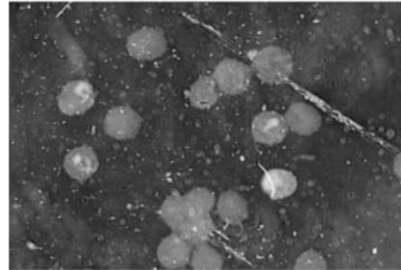
For this slide we can see how the dilation operator decreases the size of holes while erosion does the opposite. Moreover, we can see how the dilation operator thickens bridges between blobs while erosion eliminates them. In the case of the donut, we can see how important the choice of structuring element is because aside from dictating the orientation of the resultant image – it can also delete it entirely according to the rules!

Noise Removal in Binary Images

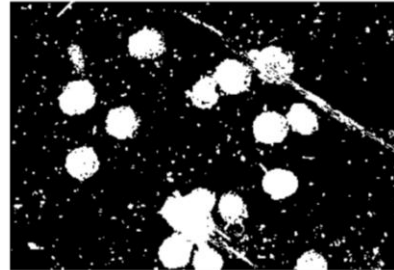
Original Image



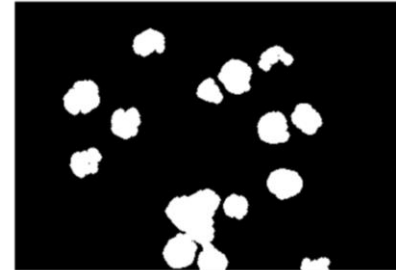
Grayscale Image



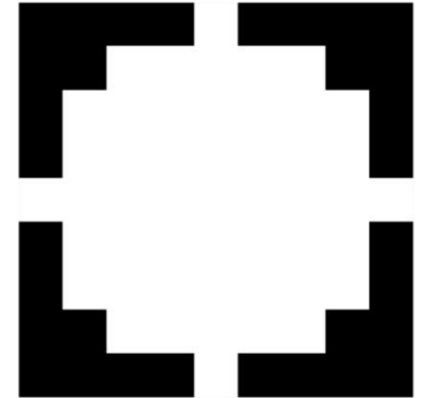
Binarized Image



Treated with Closing Operator



Structuring Element



Here, we see the reason why we study these morphological operations. We can combine dilation and erosion to produce more complicated operators to clean up binary images. In this case, the **closing** operator is a dilation followed by an erosion – which can delete pixels in an image if they are below a certain size!

When binarizing a grayscale image we can often come across artifacts due to the imperfections of thresholding. Luckily, we can now remove this noise!

In this example, we have an image of malaria cells that we would like to segment for future purposes. Converting this image to grayscale and binarizing, we get the third image with lots of noise.

Hence, we apply the closing operator and display the result in an inversed color map. Here, the structuring element used is a “disk” with radius 4 pixel. We can see in the fourth image the clean and segmented malaria cells. We are now ready for feature extraction software/programs!

Key Takeaways

- The dilation operator expands borders, closes holes, and widens bridges while the erosion operator does the opposite.
- We can combine these two basic morphological operators to create others such as the closing operator. These will enable us to remove noise when segmenting an image in preparation for feature extraction.

Reflection

I had no formal background in Set Theory back from my high school and even when I entered the university. All that I know from that field was from my own curiosity only, and so there were some gaps in my knowledge regarding this topic. The derivations for each morphological operator from first principles was honestly a fun read – and the module was quite understandable. However, while doing the morphological operations on paper was really simple, it was very tedious. I did not like this part, but it did give me a solid intuition on how the operators worked. After this, implementing them in Python was really straightforward because of the extensive libraries available.

I did all the required tasks, and wrote all the code myself since the module only had sample code written in MATLAB. For this, I think I deserve a 100/100.

References

[1] Soriano, M. (2023). Morphological Operations. App Physics 157. National Institute of Physics, UP- Diliman.